A High-Performance Document Image Binarization Pipeline with Parallel Image Processing

Aghiad Khertabeel
aghiad.khertabeel@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

Mohamad Khaled Minawe
minawe.mohamad.khaled@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

Abstract

This is a placeholder abstract for the paper.

Keywords

document image binarization, image processing, OCR preprocessing, adaptive thresholding

1 Introduction

The digitization of documents has become increasingly prevalent in various domains, including libraries, archives and businesses. The goal is to revitalize the information contained in the physical documents and make it more accessible and searchable in digital form as well as preserving it for future generations. This process involves scanning physical documents in order to create a digital representation of them. However, the quality of the scanned images can vary significantly due to factors such as lighting conditions, document physical state and the scanning equipment used. As a result, the scanned images may contain noise, shadows, and other artifacts that can hinder the process of the recognition using OCR (Optical Character Recognition) systems [5].

To address such challenges, pre-processing techniques are often applied beforehand in order to enhance the quality of the scanned images and improve the accuracy of OCR systems [4]. One of the most common pre-processing techniques is image binarization, which converts a colorful or a greyscale image into a binary image consisting solely of black and white pixels [6]. According to Gupta et al., this process allows OCR systems to focus on the relevant information in the document and ignore the background, making the OCR systems more accurate [4].

While there are many algorithms for document or image binarization available, such as Niblack's approach and Sauvola's method, these algorithms can be computationally intensive as they often require processing each pixel or each cluster of pixels individually, which can be time-consuming, especially when working with large documents with high DPI [2, 3]. Therefore, there is a growing need for high-performance solutions that entail the usage of parallel processing techniques either on the CPU level or the GPU level. Furthermore, implementing parallelization on the CPU level can be achieved by splitting the work among many worker threads, whereas with GPU-level parallelization, the work can be distributed among many GPU threads, which are usually more efficient than CPU threads when it comes generally to image processing and rendering tasks [1]. This statement was shown by Ansari et al. as they did a performance comparison between parallelized CPU and GPU performance for large metrics and the GPU performance was consistently faster than the CPU by many folds. The reason for that is because GPUs tend to have significantly more cores than CPUs, allowing them to handle a larger amount of smaller computations in parallel.

2 Background

3 System Architecture

The system implements a Scanned Document Enhancer designed for document quality improvement before printing or OCR processing. The architecture follows a modular C++ pipeline design with clear separation of concerns, enabling maintainable and testable code.

The processing utilizes sequential pipeline stages, allowing users to customize the enhancement workflow based on their specific needs.

OpenMP is also used for parallelization throughout compute-intensive operations to leverage multi-core CPU capabilities, significantly reducing processing time for large documents. The system uses the PPM P3 ASCII format for input and output images, which simplifies file handling and allows for easy debugging and visualization of intermediate results.

The design follows header/implementation separation, with clear interfaces defined in header files and implementation details encapsulated in source files. This promotes code reusability and maintainability, allowing easy extension of the system with additional enhancement techniques in the future.

3.1 Main Components

The main components can be seen in the directory structure in Listing 1:

The CLI layer, implemented in `main.cpp`, serves as the entry point of the application and the pipeline orchestration module. This layer deals with parsing arguments and configuration management.

The IO layer, located in the `io` directory, is responsible for reading and writing PPM images. It includes modules for handling the PPM P3 ASCII format, such as `ppm_reader` and `ppm_writer`, as well as a `token_reader`. The `ppm_reader` and `ppm_writer` modules handle the specifics of reading and writing PPM files, while the `token_reader` module provides utility functions for parsing the ASCII format.

The Image Processing layer, located in the `image` directory, contains `GrayImage` as the main data structure for representing grayscale images. It contains the width, height, maxval, and flat vector data, along with colorspace utilities for converting between RGB and grayscale color spaces.

The Operations layer, located in the `ops` directory, contains various image enhancement techniques, including denoising (median filter), background estimation and removal, contrast stretching, thresholding methods (Otsu's, Sauvola's, Nick's, Su's, and a proposed method), morphology operations (open and close), and border cleanup. Each operation is implemented in its own module, allowing modularity and ease of maintenance.

Several operation modules utilize integral images for efficient computation of local statistics, while others rely on direct neighborhood processing. Additionally, OpenMP is used for parallelization to ensure high performance even when processing large documents.

The Utility Layer in util directory provides `args_parser` for command-line argument processing, logging for structured output and debugging, timing for performance measurement utilities, and clamp for mathematical helper functions. The Testing Framework in tests/ uses Catch2-based testing with 56 test cases and 1038 assertions, covering component testing for I/O and algorithms as well as integration testing, with validation for algorithm correctness and edge case handling.

Listing 1: Project directory structure

```
project/
|-- CMakeLists.txt
|-- data/
|   |-- in.ppm
|   `-- out.ppm
|-- src/
|   |-- main/
|   |   `-- main.cpp
|   |-- image/
|   |   |-- image_gray.h/.cpp
|   |   `-- colorspace.h/.cpp
|   |-- io/
|   |   |-- ppm_reader.h/.cpp
|   |   |-- ppm_writer.h/.cpp
|   |   `-- token_reader.h/.cpp
|   |-- ops/
|   |   |-- denoise_median.h/.cpp
|   |   |-- background_estimate.h/.cpp
|   |   |-- background_remove.h/.cpp
|   |   |-- contrast_stretch.h/.cpp
|   |   |-- threshold_otsu.h/.cpp
|   |   |-- threshold_sauvola.h/.cpp
|   |   |-- threshold_nick.h/.cpp
|   |   |-- threshold_su.h/.cpp
|   |   |-- threshold_proposed.h/.cpp
|   |   |-- morphology.h/.cpp
|   |   `-- border_cleanup.h/.cpp
|   `-- util/
|       |-- args_parser.h/.cpp
|       |-- logging.h/.cpp
|       |-- timing.h/.cpp
|       `-- clamp.h
`-- tests/
    |-- io/
    |   `-- test_ppm_io.cpp
    `-- ops/
        |-- test_otsu.cpp
        |-- test_sauvola.cpp
        `-- test_median.cpp
```

3.2 Data Flow

The high-level pipeline follows the sequence:

(1) The user invokes the application via the command line, providing input and output file paths along with optional parameters for the enhancement pipeline.
(2) The CLI layer parses the arguments and configures the processing pipeline accordingly.
(3) The IO layer reads the input PPM image and converts it to a `GrayImage` format for processing.
(4) The Image Processing layer applies the selected enhancement operations sequentially, with each operation taking the output of the previous one as input. Operations may utilize integral images for efficient local computations and OpenMP for parallel processing.
(5) After processing, the final enhanced image is written back to a PPM file by the IO layer.

The detailed processing steps begin with the Input Stage performing PPM P3 file parsing and validation. Color Conversion transforms RGB to grayscale using standard luminance weights. The Preprocessing Chain includes optional median denoising with a 3×3 kernel, background estimation via separable box blur, background removal with target brightness adjustment, and percentile-based contrast stretching.

The Binarization Stage applies one of five configurable methods to the preprocessed image. The Postprocessing Chain performs optional morphological operations using a 3×3 structuring element and border cleanup with either fixed width or adaptive dark pixel removal. The Output Stage writes the final binary image as PPM P3 format.

Memory Management utilizes single-pass processing to minimize memory footprint, in-place operations where algorithmically feasible, integral images precomputed for adaptive thresholding methods, and OpenMP parallelization with

thread-local variables and reduction operations. This architecture ensures modularity, testability, and performance while maintaining code clarity and extensibility.

4 implementation

In this section, we will describe the implementation of all mentioned methods in the Processing layer and Operation layer of the system, starting with the denoising method, then background estimation and removal, followed by the binarization method, and finally the morphological operations. We will also discuss the parallelization techniques used in each method and how they contribute to the overall performance of the system.

4.1 Median Filter for Image Denoising

For the denoising method, we implemented a median filter, which is a non-linear digital filtering technique. The median filter works by replacing each pixel value with the median value of the pixels in its 3×3 neighborhood. This method is effective in removing salt-and-pepper noise while preserving image edges.

In our implementation, we iterate over every pixel, collect the values of its neighboring pixels, compute the median using std::nth_element, and assign it to the output image. For border pixels, the coordinates are clamped so that the nearest valid edge pixels are reused.

To improve performance, the filter is parallelized with OpenMP, allowing multiple image rows to be processed simultaneously by different threads.

4.2 Background Estimation

To estimate the background of the image, we implemented a separable box blur filter. The box blur computes the average intensity of pixels within a square neighborhood defined by a radius r. Instead of computing a full two-dimensional convolution, the filter is implemented as two one-dimensional passes: a horizontal pass followed by a vertical pass. This technique significantly reduces computational complexity.

Formally, the horizontal pass computes an intermediate image

$$I_{\text{tmp}}(x, y) = \frac{1}{2r+1} \sum_{i=-r}^{r} I\big(\text{clamp}(x+i), y\big),$$

where:

- $I(x, y)$ is the intensity of the original image at position (x, y),
- $I_{\text{tmp}}(x, y)$ is the intermediate image produced after the horizontal blur,
- r is the radius of the blur window,
- $2r + 1$ is the width of the averaging window,
- $\text{clamp}(\cdot)$ restricts the coordinate to valid image boundaries.

The vertical pass computes the final background image as

$$I_{\text{background}}(x, y) = \frac{1}{2r+1} \sum_{j=-r}^{r} I_{\text{tmp}}\big(x, \text{clamp}(y+j)\big).$$

where:

- $I_{\text{background}}(x, y)$ is the estimated background intensity,
- $I_{\text{tmp}}(x, y)$ is the horizontally blurred intermediate image,
- r is the blur radius,
- $\text{clamp}(\cdot)$ ensures that coordinates remain within valid image bounds.

Equivalently, this corresponds to averaging over a square neighborhood of size $(2r + 1) \times (2r + 1)$:

$$I_{\text{background}}(x, y) = \frac{1}{(2r+1)^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} I\big(\text{clamp}(x+i), \text{clamp}(y+j)\big).$$

where:

- (x, y) denotes the pixel position,
- $I(x, y)$ is the intensity of the original image,
- r is the blur radius,
- $(2r + 1)^2$ represents the total number of pixels in the square neighborhood.

In the horizontal pass, a sliding window of size $(2r+1)$ moves across each row of the image. The sum of the pixel values inside the window is updated efficiently by subtracting the value leaving the window and adding the value entering the window. The average value of the window is then written to a temporary image.

In the vertical pass, the same sliding-window approach is applied column-wise to the temporary image to produce the final blurred background image.

For border pixels, coordinates are clamped so that the nearest valid edge pixels are reused. To improve performance, both passes are parallelized using OpenMP, allowing multiple rows or columns to be processed simultaneously by different threads.

4.3 Background Removal

After estimating the background, it is removed from the denoised image through pixel-wise subtraction. For each pixel, the background intensity is subtracted from the denoised image intensity, and a constant target value is added to maintain a suitable brightness level.

Formally, the output pixel value is computed as

$$I_{\text{out}}(x,y) = \text{clamp}\big(I_{\text{denoised}}(x,y) - I_{\text{background}}(x,y) + T\big),$$

where:

- $I_{\text{denoised}}(x,y)$ is the intensity of the denoised image,
- $I_{\text{background}}(x,y)$ is the estimated background intensity,
- T is a constant target intensity added to maintain brightness,
- $I_{\text{out}}(x,y)$ is the final output image after background removal,
- clamp$(\cdot)$ restricts the result to the valid intensity range of the image.

This operation highlights foreground structures by removing slowly varying background illumination while ensuring that pixel values remain within the allowed intensity range.

4.4 Contrast Stretching

Contrast stretching is an image enhancement technique used to improve the contrast of a grayscale image by expanding the range of intensity values. In many images, the pixel intensities occupy only a small portion of the available dynamic range, which results in a low-contrast appearance. Contrast stretching addresses this problem by applying a linear transformation that maps the minimum and maximum intensity values of the image to the full available range.

In our implementation, we first determine the minimum and maximum pixel intensity values present in the image, denoted by $I_{\min}$ and $I_{\max}$. Each pixel value I is then transformed using the following linear scaling function:

$$I' = \frac{I - I_{\min}}{I_{\max} - I_{\min}} \times I_{\text{maxval}}$$

where:

- I is the original pixel intensity,
- I' is the transformed pixel intensity,
- $I_{\min}$ is the minimum intensity value in the image,
- $I_{\max}$ is the maximum intensity value in the image,
- I_{maxval} is the maximum possible intensity value (e.g., 255 for an 8-bit image).

This transformation linearly stretches the intensity values so that the smallest value becomes 0 and the largest value becomes I_{maxval}. As a result, the full dynamic range of the image is utilized, which increases the visual contrast.

In addition to the standard contrast stretching method, we also implement a percentile-based variant. Instead of using the absolute minimum and maximum intensity values, lower and upper thresholds are determined using selected percentiles of the image histogram. This approach reduces the influence of outlier pixels and produces a more robust contrast enhancement.

4.5 Binarization methods

The binariation phase is responsable for converting the preprocessed grayscale image into a binary image consisting of only black and white pixels. In our implementation, we provied serval methods for binarzation, including Otsu's method, Sauvola's method, Nick's method, Su's method, and a proposed method. these methods Implemented as described in the paper, but on methods such as proposed and su we applied some changes to the original method in order to make it more efficient for some use cases. These modifcations will be discuessed in the next sections.

4.5.1 Otsu's Method. In our implementation, Otsu's method is used as a global thresholding technique to convert the grayscale image into a binary image. The algorithm first computes the histogram of the image intensities. Each histogram entry counts the number of pixels having a specific intensity value.

Using this histogram, the algorithm iterates over all possible threshold values t in the intensity range $[0, I_{\max}]$. For each candidate threshold, the pixels are divided into two classes: background and foreground.

The number of pixels belonging to each class is computed as

$$w_B(t) = \sum_{i=0}^{t} h(i), \qquad w_F(t) = N - w_B(t),$$

where:

- $h(i)$ is the histogram value at intensity level i,
- $w_B(t)$ is the number of pixels in the background class,
- $w_F(t)$ is the number of pixels in the foreground class,
- N is the total number of pixels in the image.

The mean intensity of each class is then calculated as

$$\mu_B(t) = \frac{\sum_{i=0}^{t} i \cdot h(i)}{w_B(t)}, \qquad \mu_F(t) = \frac{\sum_{i=t+1}^{I_{\max}} i \cdot h(i)}{w_F(t)}.$$

Using these values, the between-class variance is computed as

$$\sigma_B^2(t) = w_B(t)\, w_F(t)\, (\mu_B(t) - \mu_F(t))^2.$$

The algorithm selects the threshold t^* that maximizes the between-class variance:

$$t^* = \arg\max_t \sigma_B^2(t).$$

After the optimal threshold is found, the binarization step is applied pixel-wise to the image. Each pixel is assigned either black or white depending on its intensity relative to the threshold:

$$I_{\text{out}}(x,y) = \begin{cases} 0, & I(x,y) \le t^* \\ I_{\max}, & I(x,y) > t^* \end{cases}$$

where:

- $I(x,y)$ is the original pixel intensity,
- $I_{\text{out}}(x,y)$ is the resulting binary pixel value,
- t^* is the optimal threshold computed by Otsu's method,
- $I_{\max}$ is the maximum intensity value (e.g., 255).

In the implementation, the final binarization step is parallelized using OpenMP, allowing multiple pixels to be processed simultaneously.

4.5.2 Implementation of NICK's Method. In our implementation, NICK's method is used as a local adaptive thresholding technique. Unlike global thresholding methods, it computes a separate threshold for each pixel based on the statistics of its local neighborhood. The input image is first validated by checking the image dimensions, the maximum intensity value, the data size, and the window radius.

To efficiently compute local statistics, two integral images are constructed: one for the sum of pixel intensities and one for the sum of squared intensities. These integral images allow the sum and squared sum over any rectangular window to be obtained in constant time.

For a pixel at position (x,y), a local window of radius r is defined around the pixel. The window boundaries are clamped to the valid image region. Let the window corners be (x_0, y_0) and (x_1, y_1). The number of pixels inside the window is given by

$$A = (x_1 - x_0 + 1)(y_1 - y_0 + 1),$$

where:

- A is the area of the local window,
- (x_0, y_0) and (x_1, y_1) are the top-left and bottom-right coordinates of the local neighborhood.

Using the integral image, the sum of all pixel intensities inside the local window is computed as

$$S(x,y) = \sum_{j=y_0}^{y_1} \sum_{i=x_0}^{x_1} I(i,j),$$

and the sum of squared intensities is computed as

$$S_2(x,y) = \sum_{j=y_0}^{y_1} \sum_{i=x_0}^{x_1} I(i,j)^2.$$

where:

- $I(i,j)$ is the grayscale intensity at pixel (i,j),
- $S(x,y)$ is the sum of intensities in the local window,
- $S_2(x,y)$ is the sum of squared intensities in the local window.

From these values, the local mean intensity is calculated as

$$m(x,y) = \frac{S(x,y)}{A},$$

and the local variance is computed as

$$\sigma^2(x,y) = \frac{S_2(x,y)}{A} - m(x,y)^2.$$

The local standard deviation is then obtained by

$$\sigma(x,y) = \sqrt{\sigma^2(x,y)}.$$

where:

- $m(x,y)$ is the local mean intensity,
- $\sigma^2(x,y)$ is the local variance,
- $\sigma(x,y)$ is the local standard deviation.

The threshold in NICK's method is computed as

$$T(x,y) = m(x,y) + k \cdot \sigma(x,y),$$

where:

- $T(x,y)$ is the local threshold for pixel (x,y),
- k is a user-defined parameter that controls the threshold shift.

In our implementation, the parameter k is typically chosen as a negative value, which lowers the threshold and improves the detection of darker foreground structures.

Finally, the output binary image is generated by comparing each pixel value with its local threshold:

$$I_{\text{out}}(x,y) = \begin{cases} 0, & I(x,y) \le T(x,y) \\ I_{\max}, & I(x,y) > T(x,y) \end{cases}$$

where:

- $I(x,y)$ is the original pixel intensity,
- $I_{\text{out}}(x,y)$ is the resulting binary pixel value,
- $I_{\max}$ is the maximum possible intensity value of the image.

To improve performance, the binarization step is parallelized using OpenMP, so that multiple image rows can be processed simultaneously. The use of integral images reduces the cost of computing local sums and variances, making the method efficient even for larger window sizes.

References

[1] Mufakir Qamar Ansari and Mudabir Qamar Ansari. Accelerating matrix multiplication: A performance comparison between multi-core cpu and gpu, 2025.

[2] Ankush Mittal Debashish Ghosh Brij Mohan Singh, Rahul Sharma. Parallel implementation of niblackís binarization approach on cuda. *International Journal of Computer Applications*, 32(2):22–27, October 2011.

[3] Xin Chen, Yuefang Gao, and Zhonghong Huang. Cuda-accelerated fast sauvola's method on kepler architecture. *Multimedia Tools Appl.*, 74(24):11809–11820, December 2015.

[4] Maya R. Gupta, Nathaniel P. Jacobson, and Eric K. Garcia. Ocr binarization and image pre-processing for searching historical documents. *Pattern Recognition*, 40(2):389–397, 2007.

[5] Mohamed Ali Souibgui and Yousri Kessentini. De-gan: A conditional generative adversarial network for document enhancement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1180–1191, March 2022.

[6] Chris Tensmeyer and Tony Martinez. Historical document image binarization: A review. *SN Comput. Sci.*, 1(3), May 2020.

A Research Methods

A.1 Part One

Placeholder for Part One.

A.2 Part Two

Placeholder for Part Two.

B Online Resources

Placeholder for Online Resources.